

Week 2 Bonus: Units and AC Phasors

Optional: worth 1/3 point on the final course grade — Separate from HW 2 Core

Optional bonus: units and AC phasors

This bonus is optional, is not a prerequisite for later work, and is not examined. It is worth **one-third of a final-grade point**. Before you start, read this week's lecture together with the self-contained reading below, which develops the complex-number and phasor background the task uses; the optional *Going further: total internal reflection* companion is not needed here.

The self-contained reading develops complex numbers, magnitude, phase, conjugation, and Euler's relation before using `cis`. We choose peak amplitudes and the time convention $\exp(+i\omega t)$, set the voltage phase to zero, build $Z = |Z| \exp(i\phi_Z)$, and calculate $I = V/Z$. The measurable current is the real part of $I \exp(i\omega t)$; the complex number is a representation, not an additional measured current.

We follow the supplied Unitful examples, verify that voltage divided by resistance has current units, and check a resistor with zero phase before trying a quarter-cycle phase. We then submit the fragment assigning the complex current in amperes and run the published equivalent-unit and invalid-input checks. We explain why a positive impedance phase gives a negative current phase. This is a single-frequency linear steady-state model; switching transients and nonlinear components are outside its scope.

Scope

This reading introduces complex numbers and Unitful quantities, then applies them to sinusoidal circuits. No previous knowledge of complex numbers or AC circuits is assumed. Allow 10–15 minutes for the reading before attempting the separate fragment. This bonus is optional, not examined, and not a prerequisite; core work does not depend on this file.

1. Complex numbers from the beginning

The imaginary unit i is defined by $i^2 = -1$. A complex number has the form

$$z = a + ib,$$

where a and b are real numbers. We call $a = \text{Re}(z)$ the real part and $b = \text{Im}(z)$ the imaginary part. In Julia the imaginary unit is written `im`, so `3 + 4im` represents $3 + 4i$.

We may draw $z = a + ib$ as the point (a, b) , or as an arrow from the origin to that point, in the complex plane. Its magnitude and phase angle are

$$|z| = \sqrt{a^2 + b^2}, \quad \phi = \text{atan2}(b, a).$$

The complex conjugate is $z^* = a - ib$, and $zz^* = a^2 + b^2 = |z|^2$. In Julia, `real(z)`, `imag(z)`, `abs(z)`, `angle(z)`, and `conj(z)` compute these quantities.

Addition combines real and imaginary parts separately. Multiplication is more interesting because $i^2 = -1$:

$$(a + ib)(c + id) = (ac - bd) + i(ad + bc).$$

Euler's relation gives a geometric form,

$$e^{i\phi} = \cos \phi + i \sin \phi.$$

Julia writes the right-hand side as `cis( $\phi$ )`. It has magnitude one and phase ϕ . Every nonzero complex number can therefore be written

$$z = |z| \operatorname{cis}(\phi).$$

Multiplication adds phase angles; division subtracts them. For example,

$$\frac{6 \operatorname{cis}(0)}{2 \operatorname{cis}(\pi/2)} = 3 \operatorname{cis}(-\pi/2) = -3i.$$

The same calculations are quick to try in Julia. The listing computes the parts, magnitude, phase, and conjugate of one number, then adds, multiplies, and divides it by another:

```
z = 3 + 4im          # a complex number with a = 3, b = 4
w = 2 - 1im
println(real(z), ", ", imag(z))  # real and imaginary parts
println(abs(z))                  # magnitude |z|
println(angle(z))                # phase in radians
println(conj(z))                 # complex conjugate
println(z + w)                   # add: parts combine separately
println(z * w)                   # multiply
println(z / w)                   # divide
println(6cis(0) / (2cis(pi/2)))  # polar division
```

```
3, 4
5.0
0.9272952180016122
3 - 4im
5 + 3im
10 + 5im
0.4 + 2.2im
1.8369701987210297e-16 - 3.0im
```

The last line is $-3i$ up to a small floating-point error in $\cos(\pi/2)$; results like this are compared with a tolerance rather than with exact equality. The magnitude and phase reproduce $z = |z| \operatorname{cis}(\phi)$: here $|z| = 5$ and $\phi \approx 0.927$ rad.

Complex numbers are two real numbers with useful geometry

An imaginary component is not automatically an imaginary physical quantity. Complex notation stores an amplitude and phase compactly. We recover the measurable real signal at the end.

2. Sinusoids and phasors

A sinusoidal voltage can be written

$$v(t) = V_0 \cos(\omega t + \phi_V),$$

where $V_0 \geq 0$ is peak voltage in volts, ω is angular frequency in rad/s, t is time in seconds, and ϕ_V is phase in radians. We use the sign convention $e^{+i\omega t}$ and define the voltage phasor

$$\tilde{V} = V_0 \operatorname{cis}(\phi_V).$$

It omits the common time factor. The physical voltage is recovered through

$$v(t) = \operatorname{Re}[\tilde{V} e^{i\omega t}].$$

Thus the phasor is a calculation device, not an extra measurable imaginary voltage. Its magnitude stores peak amplitude and its argument stores phase.

In this exercise voltage is the phase reference, so $\phi_V = 0$ and $\tilde{V} = V_0$. A positive phase means a cosine reaches the same point in its cycle earlier under our convention. Changing to $e^{-i\omega t}$ would reverse all phase signs, so stating the convention is essential.

3. AC Ohm's law and impedance

In a linear circuit driven at one angular frequency, resistance and reactive effects are combined into the complex impedance

$$\tilde{Z} = |Z| \text{cis}(\phi_Z).$$

Its magnitude $|Z| > 0$ is measured in ohms and ϕ_Z is its phase. Sinusoidal steady-state Ohm's law is

$$\tilde{I} = \frac{\tilde{V}}{\tilde{Z}} = \frac{V_0}{|Z|} \text{cis}(-\phi_Z).$$

The current amplitude is therefore $V_0/|Z|$, while its phase relative to the reference voltage is $-\phi_Z$.

For a pure resistor, $\phi_Z = 0$, so voltage and current are in phase and this reduces to ordinary Ohm's law. If $\phi_Z = \pi/2$, then current has phase $-\pi/2$ and lags the voltage by one quarter-cycle. The unit check is

$$\frac{\text{V}}{\Omega} = \frac{\text{V}}{\text{V/A}} = \text{A}.$$

A 12 V source driving a 4 Ω impedance at phase $\pi/2$ illustrates the computation:

```
V = 12cis(0)          # voltage phasor: 12 V at zero phase
Z = 4cis(pi/2)        # impedance: 4 ohm at phase pi/2
I = V / Z              # current phasor
println(round(abs(I), digits=3), " A at phase ",
        round(angle(I), digits=3), " rad")
```

3.0 A at phase -1.571 rad

The current amplitude is $V_0/|Z| = 3$ A, and its phase is $-\phi_Z \approx -\pi/2$ rad, so the current lags the voltage by a quarter-cycle.

Physical model and limits

An alternating source drives charges back and forth. Resistance dissipates energy, while ideal capacitive or inductive storage can shift current relative to voltage. Complex impedance summarizes amplitude ratio and phase shift at one frequency. We use peak amplitudes and the $e^{+i\omega t}$ convention, with voltage as the zero-phase reference. Known checks are $12 \text{ V}/4 \Omega = 3 \text{ A}$ at zero phase and $-3i \text{ A}$ at impedance phase $\pi/2$. This model assumes a linear circuit in sinusoidal steady state at one fixed frequency. It omits switch-on transients, harmonics, nonlinear components, parameter changes caused by heating, and frequency dependence not already included in the supplied impedance.

4. Units in Julia

The `Unitful` package attaches units to values:

```
using Unitful

voltage = 12u"V"
impedance_magnitude = 4u"Ω"
phase = 30u"°"
impedance = impedance_magnitude * cis(phase)
current = uconvert(u"A", voltage / impedance)
```

The resulting current is approximately $(2.598 - 1.500i)$ A. Unitful rejects incompatible conversions, while `uconvert` expresses a compatible result in the requested unit. Angles must carry a Unitful angular unit in this task, even though radians are mathematically dimensionless.

Optional status

This bonus is optional, is not a prerequisite for later work, and is not examined.

Task

The assessment environment loads Unitful and supplies this wrapper:

```
function ac_current(voltage_amplitude,
                   impedance_magnitude,
                   impedance_phase)
    # your submitted fragment is inserted here
    return current
end
```

The inputs must be Unitful quantities. Voltage amplitude must have voltage dimensions and be finite and nonnegative. Impedance magnitude must have impedance dimensions and be finite and strictly positive. Impedance phase must be a finite Unitful angle. Invalid input must throw `ArgumentError`. Assign `current` to the complex current converted to amperes and print nothing.

Use `Unitful.dimension(x)` to compare dimensions with `Unitful.dimension(1u"V")` or `Unitful.dimension(1u"Ω")`. A Unitful angle has `Unitful.NoDims`. After dimensions are checked, `Unitful.ustrip(u"V", x)` or the corresponding unit yields a plain number for a finiteness check.

Allowed technique

Use scalar values, complex arithmetic, `cis`, Unitful operations, comparisons, and explicit `if/elseif` validation. Do not define a function, use loops or collections, install packages, access the network, or print.

Exact student template

```
# Validate all three Unitful inputs, construct impedance,
# and assign current.
current = 0u"A" # A
```

How is this exercise checked?

The public test body below shows exactly what is checked. Each `@test` states a claim that must hold for our fragment, and each `@test_throws` `ArgumentError` states that a documented invalid input must be rejected; running them gives the same verdict every time.

```
using Test
using Unitful

@test ac_current(12u"V", 4u"Ω", 0u"rad") == 3u"A"
@test isapprox(
    ac_current(12u"V", 4u"Ω", (pi / 2)u"rad"),
    -3im * u"A"; atol=1e-12u"A")
@test isapprox(
    ac_current(12u"V", 4u"Ω", 30u"°"),
    (3cos(pi / 6) - 3sin(pi / 6)im)u"A";
    atol=1e-12u"A")
@test Unitful.dimension(
    ac_current(500u"mV", 2u"Ωk", 0u"rad")) ==
    Unitful.dimension(1u"A")
@test isapprox(ac_current(500u"mV", 2u"Ωk", 0u"rad"),
    0.25u"mA"; atol=1e-12u"A")
@test_throws ArgumentError ac_current(12u"V", 0u"Ω", 0u"rad")
@test_throws ArgumentError ac_current(12u"V", 4u"Ω", 1u"s")
```

The staff grader tests use this same documented contract. They may vary the valid inputs and the documented error cases, but they add no new requirements.

Expected submission

Submit only the fragment, separately from HW 2 core. Do not include the supplied wrapper or package-installation commands.

Advanced (optional)

The wrapper and the `@test` statements belong to the platform, not to us. A wrapper is a supplied function that calls our fragment and returns its result—here, `current`—while `@test` states a claim that must hold and `@test_throws` states that an invalid input must be rejected. We do not write functions for this optional Week 2 fragment; reading the tests simply shows us how the fragment is checked reproducibly.